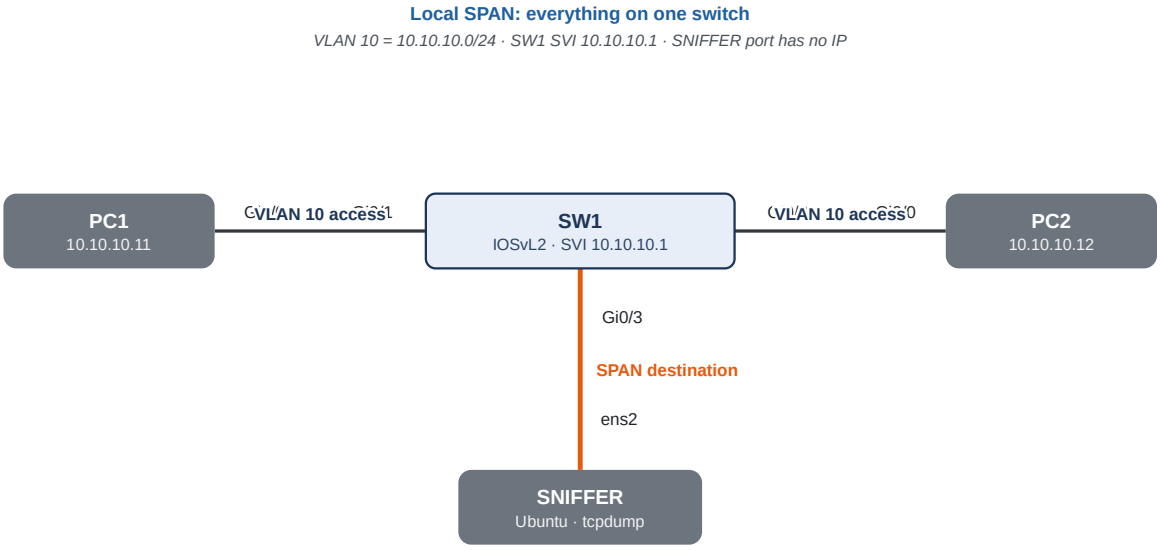
# SPAN Lab 01: Local SPAN

## Lab Objective:

The focus of this lab is to understand local Switched Port Analyzer (SPAN) sessions in Cisco IOS switches. Additional technologies include SPAN source ports and VLANs, traffic direction (rx, tx, both), the SPAN destination port, and SPAN session limits.

## Lab Topology:

The lab network topology is illustrated below:




*Copies of the monitored traffic leave Gi0/3 (orange). The sniffer only listens.*

## IMPORTANT NOTE

Use IOSvL2 for SW1, IOSv for PC1 and PC2 (with `ip routing` and a default route to 10.10.10.1), and an Ubuntu node as SNIFFER. Check the sniffer's interface name with `ip link` (usually `ens2`).

You can watch the mirrored traffic in two ways: run `sudo tcpdump -n -i ens2` on SNIFFER, or start a CML packet capture on the SW1–SNIFFER link. On IOSvL2, SPAN works for normal traffic, but it's an emulator: if a capture stays empty although `show monitor` looks right, restart the capture before you suspect your config.

## Task 1

Configure VLAN 10 on SW1 and put Gi0/1 and Gi0/2 into it as access ports. Give SW1 an SVI in VLAN 10 with the address 10.10.10.1/24. Configure PC1 and PC2. Verify that PC1 can ping PC2 and 10.10.10.1. Start a capture on SNIFFER and verify that it sees no unicast traffic between PC1 and PC2. Explain why.

## Task 2

Configure SPAN session 1 on SW1 to copy all traffic sent and received on Gi0/1 to Gi0/3. Verify the session, the state of Gi0/3, and the capture on SNIFFER while PC1 pings PC2.

## Task 3

Change session 1 so that it copies only the traffic that **SW1 receives** on Gi0/1. PC1 pings PC2 again. Which ICMP messages does SNIFFER see now, and why?

## Task 4

Replace the interface source with VLAN 10 as the source, so that all traffic in VLAN 10 is monitored, including PC2's traffic to the SVI. Choose the direction so that each frame is copied only once. Verify the session and the capture.

## Task 5

Try to configure Gi0/3 as a SPAN source in a second session, and Gi0/3 as an access port in VLAN 10. Note what IOS allows and what it refuses. Finally, remove session 1 and verify that Gi0/3 is a normal port again.

# Lab Validation:

The outputs below are trimmed. Counters and times will differ in your lab.

### Task 1

```
SW1(config)#vlan 10
SW1(config-vlan)#exit
SW1(config)#interface range GigabitEthernet0/1 - 2
SW1(config-if-range)#switchport mode access
SW1(config-if-range)#switchport access vlan 10
SW1(config-if-range)#exit
SW1(config)#interface Vlan10
SW1(config-if)#ip address 10.10.10.1 255.255.255.0
SW1(config-if)#no shutdown
PC1(config)#interface GigabitEthernet0/0
PC1(config-if)#ip address 10.10.10.11 255.255.255.0
PC1(config-if)#no shutdown
PC1(config-if)#exit
PC1(config)#ip route 0.0.0.0 0.0.0.0 10.10.10.1
```

PC2 is the same with 10.10.10.12.

```
sniffer$ sudo tcpdump -n -i ens2
```

While PC1 pings PC2, SNIFFER sees nothing (or only broadcasts, such as ARP or STP). A switch forwards a unicast frame **only out of the port where the destination MAC lives**. The sniffer isn't the destination, so it never gets a copy. That's why a switched network needs SPAN for packet captures: on an old hub, every port saw everything.

### Task 2

```
SW1(config)#monitor session 1 source interface GigabitEthernet0/1 both
SW1(config)#monitor session 1 destination interface GigabitEthernet0/3
```

```
SW1#show monitor session 1
Session 1
---------
```

```
Type                      : Local Session
Source Ports              :
    Both                  : Gi0/1
Destination Ports         : Gi0/3
    Encapsulation         : Native
          Ingress         : Disabled

SW1#show interfaces GigabitEthernet0/3 | include line protocol
GigabitEthernet0/3 is up, line protocol is up (monitoring)

sniffer$ sudo tcpdump -n -i ens2 icmp
IP 10.10.10.11 > 10.10.10.12: ICMP echo request, id 1, seq 0, length 80
IP 10.10.10.12 > 10.10.10.11: ICMP echo reply, id 1, seq 0, length 80
```

| Part | Meaning |
|---|---|
| `source interface ... both` | Copy frames the switch **receives** (rx) and **sends** (tx) on that port. `both` is the default |
| `destination interface` | The port where the copies leave, towards the analyzer |
| `(monitoring)` | Gi0/3 is now a SPAN destination: it doesn't learn MACs, doesn't take part in STP, and by default **drops everything the sniffer sends** |
| `Ingress: Disabled` | The analyzer can't send traffic into the network through this port. `ingress` can allow it, but it's rarely needed |

## Task 3

```
SW1(config)#no monitor session 1 source interface GigabitEthernet0/1
SW1(config)#monitor session 1 source interface GigabitEthernet0/1 rx

sniffer$ sudo tcpdump -n -i ens2 icmp
IP 10.10.10.11 > 10.10.10.12: ICMP echo request, id 2, seq 0, length 80
IP 10.10.10.11 > 10.10.10.12: ICMP echo request, id 2, seq 1, length 80
```

Only the **echo requests** remain. **rx and tx are always from the switch's point of view**: rx on Gi0/1 means "frames SW1 receives on Gi0/1", which is what PC1 sends. The replies from PC2 *leave* SW1 through Gi0/1, which is tx, and that's no longer monitored.

## Task 4

```
SW1(config)#no monitor session 1 source interface GigabitEthernet0/1
SW1(config)#monitor session 1 source vlan 10 rx

SW1#show monitor session 1
Session 1
---------
Type                      : Local Session
Source VLANs              :
    RX Only               : 10
Destination Ports         : Gi0/3
    Encapsulation         : Native
          Ingress         : Disabled
```

Now pings from PC2 to 10.10.10.1 show up too: every port in VLAN 10 is a source.

**rx is the right direction for a VLAN source.** A frame from PC1 to PC2 is *received* on Gi0/1 and *sent* on Gi0/2, and both ports are in VLAN 10. With `both`, SW1 would copy it **twice**, once as rx and once as tx, and

every packet would appear doubled in the capture. With `rx`, every frame that enters the VLAN is copied exactly once.

Interface sources and VLAN sources **can't be mixed** in one session. That's why the interface source was removed first.

## Task 5

```
SW1(config)#monitor session 2 source interface GigabitEthernet0/3
SW1(config)#interface GigabitEthernet0/3
SW1(config-if)#switchport access vlan 10
```

IOS refuses the first command with an error: **a port can't be a SPAN destination and a source at the same time**. The second command is accepted into the config, but it has no effect while Gi0/3 is a SPAN destination. The port keeps its monitoring role and stays out of VLAN 10. Other rules you can test the same way:

- A destination port belongs to only **one** session.
- The number of sessions per switch is limited by the platform (often 2 local/RSPAN source sessions on access switches).

```
SW1(config)#no monitor session 2
SW1(config)#no monitor session 1
SW1#show interfaces GigabitEthernet0/3 | include line protocol
GigabitEthernet0/3 is up, line protocol is up (connected)
```

After `no monitor session 1`, Gi0/3 is a normal switch port again, and its configured VLAN 10 now takes effect.

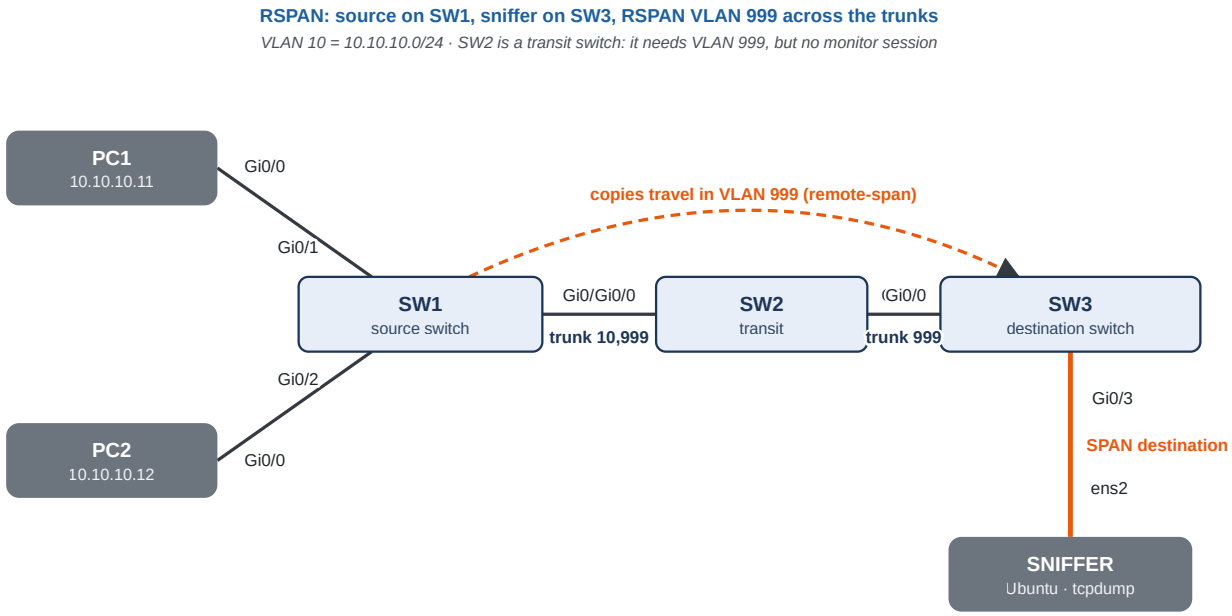
# SPAN Lab 02: Remote SPAN (RSPAN)

## Lab Objective:

The focus of this lab is to understand Remote SPAN (RSPAN) in Cisco IOS switches. Additional technologies include the RSPAN VLAN, source and destination sessions on different switches, transit switches, MAC learning in the RSPAN VLAN, and trunk pruning.

## Lab Topology:

The lab network topology is illustrated below:




## IMPORTANT NOTE

Use IOSvL2 for SW1, SW2 and SW3, IOSv for PC1 and PC2 (with `ip routing` and a default route to 10.10.10.1), and an Ubuntu node as SNIFFER. Run `vtp mode transparent` on all three switches so that each switch's VLAN configuration stays local and you configure every VLAN yourself.

RSPAN solves one problem: the sniffer is on a **different switch** than the traffic you want to see. The copies travel to it at Layer 2, in a dedicated VLAN.

## Task 1

Configure VLAN 10 on SW1 with Gi0/1 and Gi0/2 as access ports, and an SVI with 10.10.10.1/24. Configure PC1 and PC2. Configure the links SW1–SW2 and SW2–SW3 as 802.1Q trunks. Verify that PC1 can ping PC2.

## Task 2

Create VLAN 999 as an RSPAN VLAN on every switch that the mirrored traffic passes through. Verify the RSPAN VLAN on all three switches.

## Task 3

Configure SW1 to send a copy of all traffic sent and received on Gi0/1 into the RSPAN VLAN. Configure SW3 to deliver the traffic arriving in the RSPAN VLAN to the sniffer on Gi0/3. SW2 gets no monitor session. Verify both sessions, and the capture on SNIFFER while PC1 pings PC2.

## Task 4

Verify the MAC address table for VLAN 999 on SW2 and SW3, and explain the result. Then restrict the trunks so that only the VLANs that need them cross each link: VLANs 10 and 999 on SW1–SW2, and VLAN 999 only on SW2–SW3. Verify that the capture still works.

# Lab Validation:

The outputs below are trimmed. Counters and times will differ in your lab.

### Task 1

```
SW1(config)#vtp mode transparent
SW1(config)#vlan 10
SW1(config-vlan)#exit
SW1(config)#interface range GigabitEthernet0/1 - 2
SW1(config-if-range)#switchport mode access
SW1(config-if-range)#switchport access vlan 10
SW1(config-if-range)#exit
SW1(config)#interface Vlan10
SW1(config-if)#ip address 10.10.10.1 255.255.255.0
SW1(config-if)#no shutdown
SW1(config-if)#exit
SW1(config)#interface GigabitEthernet0/0
SW1(config-if)#switchport trunk encapsulation dot1q
SW1(config-if)#switchport mode trunk
```

The same trunk configuration goes on SW2 Gi0/0 and Gi0/1 and on SW3 Gi0/0. PC1 and PC2 are configured as in SPAN Lab 01.

### Task 2

On SW1, SW2 and SW3:

```
SW2(config)#vlan 999
SW2(config-vlan)#remote-span
SW2(config-vlan)#name RSPAN
```

```
SW2#show vlan remote-span
Remote SPAN VLANs
------------------------------------------------------------------------------
999
```

The `remote-span` keyword is what makes VLAN 999 special: the switches **turn off MAC learning** in it and treat it purely as a transport for mirrored frames. It must exist on the **source, transit and destination switches**. If a transit switch doesn't know VLAN 999, the copies are dropped there. VLAN 1 and 1002–1005 can't be RSPAN VLANs.

## Task 3

```
SW1(config)#monitor session 1 source interface GigabitEthernet0/1 both
SW1(config)#monitor session 1 destination remote vlan 999
SW3(config)#monitor session 1 source remote vlan 999
SW3(config)#monitor session 1 destination interface GigabitEthernet0/3
```

```
SW1#show monitor session 1
Session 1
---------
Type                    : Remote Source Session
Source Ports            :
    Both                : Gi0/1
Dest RSPAN VLAN         : 999

SW3#show monitor session 1
Session 1
---------
Type                    : Remote Destination Session
Source RSPAN VLAN       : 999
Destination Ports       : Gi0/3
    Encapsulation       : Native
          Ingress       : Disabled

sniffer$ sudo tcpdump -n -i ens2 icmp
IP 10.10.10.11 > 10.10.10.12: ICMP echo request, id 1, seq 0, length 80
IP 10.10.10.12 > 10.10.10.11: ICMP echo reply, id 1, seq 0, length 80
```

An RSPAN setup is **two sessions**, glued together by the RSPAN VLAN:

| Switch | Session | Job |
|---|---|---|
| SW1 | Remote **source** session | Copies Gi0/1 traffic and sends the copies **into VLAN 999** |
| SW2 | None | Just a switch carrying VLAN 999 over its trunks |
| SW3 | Remote **destination** session | Takes everything in **VLAN 999** and sends it out Gi0/3 |

The session numbers don't have to match between switches. Each session is local to its own switch, and only the VLAN number ties them together.

## Task 4

```
SW2#show mac address-table vlan 999
          Mac Address Table
-------------------------------------------
Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
SW2#
```

The table is **empty**, even though the mirrored frames constantly cross SW2. The copies still carry the original source MACs: PC1's and PC2's. If SW2 learned them in VLAN 999, the table would fill with addresses that don't really live there. Since every frame in an RSPAN VLAN is a copy going to one place, the switches don't learn MACs in it and simply **flood it** along the trunks. That's also why the RSPAN VLAN should only exist on the trunks along the path.

```
SW1(config)#interface GigabitEthernet0/0
SW1(config-if)#switchport trunk allowed vlan 10,999
SW2(config)#interface GigabitEthernet0/0
SW2(config-if)#switchport trunk allowed vlan 10,999
```

```
SW2(config-if)#interface GigabitEthernet0/1
SW2(config-if)#switchport trunk allowed vlan 999
SW3(config)#interface GigabitEthernet0/0
SW3(config-if)#switchport trunk allowed vlan 999

SW2#show interfaces trunk | begin allowed and active
Port        Vlans allowed and active in management domain
Gi0/0       10,999
Gi0/1       999
```

Restricting the RSPAN VLAN matters in real networks: monitoring a busy port can double that port's traffic, and an RSPAN VLAN that's allowed everywhere floods those copies over **every trunk** in the network. The capture still works, because the whole path SW1 → SW2 → SW3 carries VLAN 999.

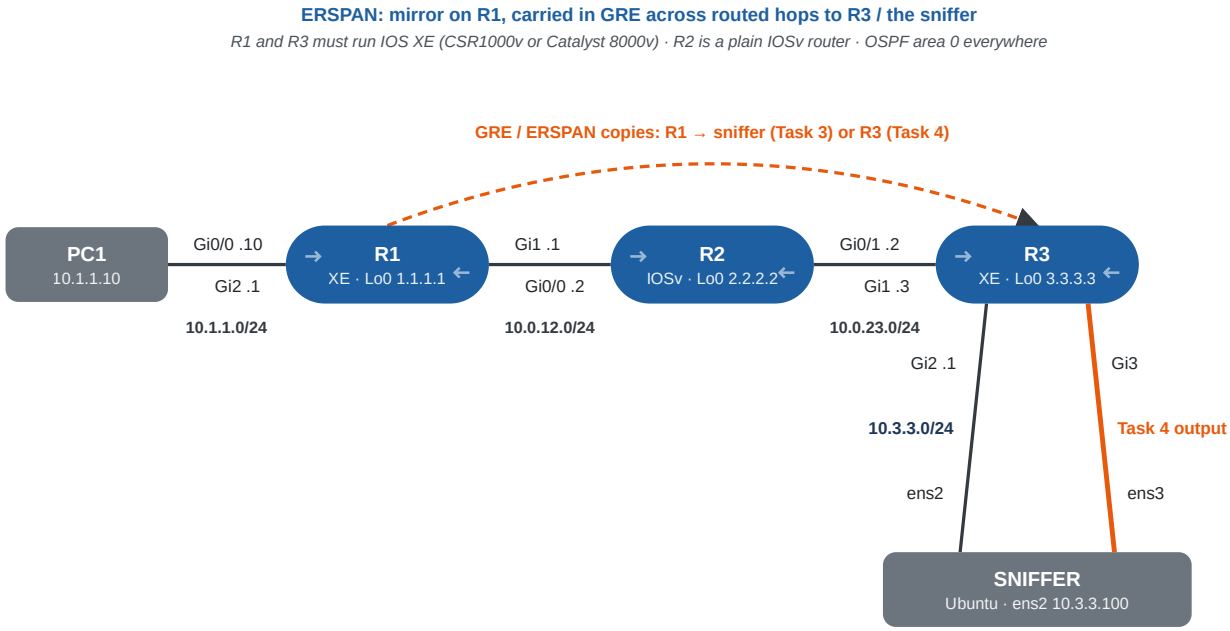
# SPAN Lab 03: Encapsulated Remote SPAN (ERSPAN)

## Lab Objective:

The focus of this lab is to understand Encapsulated Remote SPAN (ERSPAN) in Cisco IOS XE routers. Additional technologies include ERSPAN source sessions, the ERSPAN ID, GRE encapsulation across a routed network, sending ERSPAN straight to an analyzer, and terminating ERSPAN on an IOS XE destination session.

## Lab Topology:

The lab network topology is illustrated below:




## IMPORTANT NOTE

ERSPAN isn't supported on IOSv or IOSvL2. Use **CSR1000v or Catalyst 8000v** for R1 and R3. They need more RAM (about 3–4 GB each) and take a few minutes to boot. R2 is a normal IOSv router: it only routes the GRE packets and needs no ERSPAN support. On IOS XE the interfaces are named GigabitEthernet1, 2, 3 (Gi1 to the core). PC1 is IOSv with a default route to 10.1.1.1, and SNIFFER is an Ubuntu node with two interfaces.

Run OSPF area 0 on all router interfaces and loopbacks. Give SNIFFER's ens2 the address 10.3.3.100/24 ( `sudo ip addr add 10.3.3.100/24 dev ens2` , `sudo ip link set ens2 up` , `sudo ip route add default via 10.3.3.1` ). Leave ens3 without an address.

## Task 1

Configure the addressing from the topology and OSPF area 0 on R1, R2 and R3, including the loopbacks. Configure PC1. Verify that PC1 can ping 10.3.3.100 and 3.3.3.3.

## Task 2

Configure an ERSPAN source session on R1 that copies all traffic sent and received on Gi2. Use ERSPAN ID 100, send the copies to the sniffer at 10.3.3.100, and use R1's Loopback0 as the source of the ERSPAN packets. Verify the session.

## Task 3

PC1 pings 3.3.3.3. Capture on SNIFFER's ens2 and examine what arrives. What is the outer IP header, which protocol carries the copies, and where is PC1's original packet?

## Task 4

Instead of sending the ERSPAN packets to the sniffer, terminate them on R3: configure an ERSPAN destination session on R3 that receives ERSPAN ID 100 at 3.3.3.3 and sends the original frames out Gi3 to the sniffer's ens3. Change R1's session to send to R3. Verify both sessions and the capture on ens3.

# Lab Validation:

The outputs below are trimmed. Counters and times will differ in your lab.

### Task 1

```
R1(config)#interface GigabitEthernet1
R1(config-if)#ip address 10.0.12.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#interface GigabitEthernet2
R1(config-if)#ip address 10.1.1.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#interface Loopback0
R1(config-if)#ip address 1.1.1.1 255.255.255.255
R1(config-if)#exit
R1(config)#router ospf 1
R1(config-router)#router-id 1.1.1.1
R1(config-router)#network 0.0.0.0 255.255.255.255 area 0
```

R2 and R3 follow the same pattern with their addresses. R3 also has Gi2 10.3.3.1/24 toward the sniffer. **Gi3 on R3 gets no IP address**: it becomes the ERSPAN output port in Task 4.

```
PC1#ping 10.3.3.100
!!!!!
PC1#ping 3.3.3.3
!!!!!
```

### Task 2

```
R1(config)#monitor session 1 type erspan-source
R1(config-mon-erspan-src)#source interface GigabitEthernet2 both
R1(config-mon-erspan-src)#no shutdown
```

```
R1(config-mon-erspan-src)#destination
R1(config-mon-erspan-src-dst)#erspan-id 100
R1(config-mon-erspan-src-dst)#ip address 10.3.3.100
R1(config-mon-erspan-src-dst)#origin ip address 1.1.1.1
```

| Command | Meaning |
|---|---|
| `monitor session 1 type erspan-source` | ERSPAN source session (on IOS XE, every ERSPAN session has a type) |
| `source interface ... both` | What to copy. Same rx/tx/both logic as local SPAN |
| `no shutdown` | ERSPAN sessions are **shut down by default**. Forgetting this is the classic mistake |
| `erspan-id 100` | Session ID inside the ERSPAN header, so the receiver can tell different sessions apart |
| `ip address 10.3.3.100` | Where the copies go: any **routable** IP address |
| `origin ip address 1.1.1.1` | Source IP of the ERSPAN packets |

```
R1#show monitor session 1
Session 1
---------
Type                   : ERSPAN Source Session
Status                 : Admin Enabled
Source Ports           :
    Both               : Gi2
Destination IP Address : 10.3.3.100
Origin IP Address      : 1.1.1.1
ERSPAN ID              : 100
```

## Task 3

```
sniffer$ sudo tcpdump -n -i ens2 proto gre
IP 1.1.1.1 > 10.3.3.100: GREv0, seq 12, length 142: ERSPAN, ...
IP 1.1.1.1 > 10.3.3.100: GREv0, seq 13, length 142: ERSPAN, ...
```

The details depend on the tcpdump version. Open the CML capture of that link in Wireshark to see every layer decoded. Each copy looks like this, from the outside in:

| Layer | Content |
|---|---|
| Outer IP | 1.1.1.1 → 10.3.3.100, an ordinary routed packet |
| GRE | Protocol type 0x88BE = ERSPAN |
| ERSPAN header | Session ID 100, VLAN, etc. |
| Original frame | PC1's **complete Ethernet frame**: MACs, IP 10.1.1.10 → 3.3.3.3, ICMP |

That's the whole difference from RSPAN: **RSPAN** carries copies in a **Layer 2 VLAN**, so the analyzer must be in the same switched network. **ERSPAN** wraps them in **GRE/IP**, so they're **routed** and can travel across routers, a WAN, or into a data centre. R2 is just routing ordinary IP packets here.

The encapsulation adds about **50 bytes** per copy (outer IP 20 + GRE 8 + ERSPAN 8 + the copied frame's own Ethernet header). Copies of full-size packets can exceed the path MTU. Depending on the platform, they're then fragmented or truncated, so check the MTU on the ERSPAN path in real networks.

## Task 4

On R3:

```
R3(config)#monitor session 2 type erspan-destination
R3(config-mon-erspan-dst)#destination interface GigabitEthernet3
R3(config-mon-erspan-dst)#no shutdown
R3(config-mon-erspan-dst)#source
R3(config-mon-erspan-dst-src)#erspan-id 100
R3(config-mon-erspan-dst-src)#ip address 3.3.3.3
```

On R1, point the session at R3 instead of the sniffer:

```
R1(config)#monitor session 1 type erspan-source
R1(config-mon-erspan-src)#destination
R1(config-mon-erspan-src-dst)#ip address 3.3.3.3
```

```
R3#show monitor session 2
Session 2
---------
Type                  : ERSPAN Destination Session
Status                : Admin Enabled
Destination Ports     : Gi3
Source IP Address     : 3.3.3.3
Source ERSPAN ID      : 100

sniffer$ sudo tcpdump -n -i ens3 icmp
IP 10.1.1.10 > 3.3.3.3: ICMP echo request, id 3, seq 0, length 80
IP 3.3.3.3 > 10.1.1.10: ICMP echo reply, id 3, seq 0, length 80
```

On ens3 the sniffer now sees **PC1's original packets with no GRE at all**. R3 accepted the ERSPAN packets addressed to 3.3.3.3 with ID 100, removed the outer IP, GRE and ERSPAN headers, and sent the original frames out Gi3. Just like a local SPAN destination port, Gi3 is now a monitoring-only port.

Which option to choose:

- **ERSPAN straight to the analyzer** (Task 3) is the most common today: Wireshark and most analyzers decode ERSPAN themselves.
- **Terminating on a router or switch** (Task 4) is for tools that need plain frames on a port, for example an IDS sensor.

The ERSPAN ID must match on both ends: R3 only accepts sessions it's configured for.